

Problem Statement Analysis

Corporate Learning Progress, Intervention & Compliance Tracking System

Field	Details
Document Title	Problem Statement Analysis – Corporate Learning Progress, Intervention & Compliance Tracking System
Author	Taarak Aggarwal
Role	Senior Architect
Date	May 22, 2026
Version	1.2
Status	Draft

1. Problem Statement

Fragmented Learning Data

Organizations are accountable for employee learning outcomes, attendance, and compliance, yet learning data is fragmented across LMS platforms, assessment systems, spreadsheets, and trainer notes. This fragmentation makes it difficult to establish a reliable single source of truth for employee competency progression.

Late Detection of At-Risk Learners

Employees who miss training, score below thresholds, or fail to progress through competency milestones are often identified too late—typically during escalations or compliance reviews. This delayed detection reduces the effectiveness of timely remediation.

Reactive Interventions

Most interventions are triggered after a learner has already fallen significantly behind. A better solution is to detect risk earlier and support proactive remedial training, coaching, or mentoring before compliance or capability failures occur.

Compliance Reporting Gaps

Compliance reporting is often manual and error-prone because data has to be stitched together across multiple tools and sources. This increases audit risk and weakens confidence in reported learning and competency status.

2. What is Expected

The expected solution is a focused digital learning progress and early-intervention tracking platform—not a full enterprise LMS—that consolidates learning data, identifies at-risk learners early, tracks interventions, and provides compliance-ready reporting.

Primary Objectives

1. Consolidate training attendance records, periodic assessment scores, and competency-level milestones into a unified employee learning profile.
2. Implement configurable, rule-based risk identification using attendance thresholds, score thresholds, milestone due dates, and composite conditions.
3. Track interventions such as remedial training, coaching, and mentoring, including effectiveness and outcomes over time.
4. Generate compliance-ready reports that demonstrate training completion, competency attainment, risk status, and remediation history.
5. Provide practical dashboards and reports for L&D administrators, trainers, compliance teams, and HR stakeholders.

Must-Have Features

- Employee learning profile aggregation
- Configurable risk rules (attendance %, score thresholds, milestone deadlines)
- At-risk learner classification
- Intervention history and outcome tracking
- Simple dashboards or reports

3. What is in Scope

The following capabilities are explicitly within scope for the proposed platform:

Data Ingestion (via API)

- Employee training attendance records
- Periodic assessment scores

- Competency-level learning milestones

Intervention Tracking

- Remedial training sessions
- Coaching and mentoring assignments

Rule-Based Risk Identification

Define a configurable rule format that separates input dataset, threshold conditions, time windows, severity classification, and actions. Example rules include attendance below 75%, assessment score below 60%, and missed milestones beyond due-date tolerance.

Compliance-Ready Reporting

Provide reports for audits and internal reviews, including learner status, competency progression, triggered risk rules, and intervention records.

Employee Learning Profile Aggregation

Aggregate attendance, scores, milestones, risk status, and intervention history into a single employee-level view.

Dashboards and Reports

Provide practical dashboards and reports for L&D, trainers, compliance officers, and HR managers.

Clean Separation of Layers

Maintain clear separation across ingestion/data storage, rules/business logic, reporting/presentation, and access-control concerns.

4. What is NOT in Scope

- **Full Corporate LMS Build** — The system will not replace a corporate LMS or provide full course lifecycle management, enrollment workflows, or content catalogs.
- **Course Content Authoring or Delivery** — No authoring tools or direct course delivery/streaming capabilities are included.
- **E-Learning Content Hosting or Streaming** — The platform will not host learning videos or e-learning assets.
- **LMS User Authentication / SSO Integration** — Enterprise-grade identity integration may be considered in a later phase but is not core scope.

- **Gamification or Social Learning Features** — Badges, points, discussion forums, and social learning are excluded.
- **Mobile Application Development** — No dedicated native mobile application in the initial release.
- **Integration with HRIS / Payroll Systems** — HRIS and payroll integrations are excluded from the initial build.
- **Advanced AI/ML Predictions** — The system focuses on rule-based risk identification, not ML-based prediction in phase 1.
- **Real-Time Video / Virtual Classroom Management** — No virtual classroom or conferencing management capability.
- **Certificate or Credential Issuance** — No digital certificate or badge issuance.
- **Third-Party LMS Migration** — Migration of legacy LMS content/history is excluded.
- **Learner Self-Service Portal** — A learner-facing self-service portal is not part of the initial scope.

5. Future Scope

6. **AI/ML-Based Predictive Analytics** — Predict at-risk learners using historical patterns before thresholds are breached.
7. **HRIS, HRMS & Payroll Integration** — Enrich learner context with role, hierarchy, and business unit data.
8. **Mobile Application** — Enable mobile access for managers and learners in future phases.
9. **Advanced Analytics Dashboards** — Add deeper trends, benchmarks, and drill-down analytics.
10. **Automated Intervention Recommendations** — Recommend interventions based on rule triggers and historical outcomes.
11. **External Certification Body Integration** — Integrate certification validity and renewal data.
12. **Learner Self-Service Portal** — Let learners view their own progress, risk, and intervention history.
13. **Gamification & Engagement Features** — Improve engagement with badges, reminders, and achievement indicators.
14. **NLP for Trainer Notes Analysis** — Extract signals from unstructured notes and coaching feedback.
15. **Multi-Tenant SaaS Deployment** — Evolve the platform to support multiple organizations/tenants.

16. **API Marketplace for LMS Integrations** — Provide reusable connectors for common LMS vendors.
17. **Real-Time Notifications & Alerts Engine** — Send threshold-based alerts to administrators, trainers, and managers.

6. Key Stakeholders

Stakeholder	Role / Interest
L&D Administrators	Configure risk rules, manage interventions, and monitor learning/compliance progress.
Trainers / Coaches	View assigned at-risk learners and log intervention progress.
Compliance Officers	Generate and validate audit-ready reports.
HR Leadership	Monitor workforce learning health and capability gaps.
Employees / Learners	Their learning data is tracked through APIs and managed processes.
IT / Engineering Team	Build, deploy, secure, and operate the platform.

7. Evaluation Parameters

#	Evaluation Parameter	Description
1	Realism of Learning Competency Rules	Rules should reflect real-world enterprise learning and compliance scenarios.
2	Correct Handling of Competency-Level Progression	The system must model milestone progression and dependencies accurately.
3	Practical Usefulness for Trainers and L&D Administrators	The workflows should be practical, intuitive, and operationally useful.
4	Clean Separation of Data, Rules, and Reporting	Architecture should keep data management, business logic, and reporting modular.

8. Use Case Documentation

This section defines the key actors, user flows, sequence diagrams, and role-based permissions for the proposed system.

8.1 Actors

#	Actor	Type	Description
1	L&D Administrator	Primary	Configures risk rules, manages intervention programs, generates compliance reports, and oversees organization-wide learning progress.
2	Trainer / Coach	Primary	Views assigned at-risk learners, logs session updates, and tracks remedial training or coaching outcomes.
3	Compliance Officer	Primary	Reviews training completion, competency attainment, and audit-readiness reports.
4	HR Manager	Secondary	Reviews department-level dashboards and learning health trends.
5	Employee / Learner	Secondary (Passive)	Learning data is tracked, but there is no direct interaction in current scope.
6	System (Automated Engine)	System	Executes rule-based evaluation, risk classification, and system event logging.
7	External Source Systems (LMS, Assessment Platforms)	System	Provide attendance, assessment, and milestone data through APIs.

8.2 User Flows

UF-01: Employee Learning Profile View

This flow describes how an L&D Administrator or Trainer accesses a consolidated learning profile for an employee.

Field	Details
Flow ID	UF-01
Flow Name	Employee Learning Profile View

Primary Actor	L&D Administrator / Trainer
Trigger	Actor navigates to Employee Learning Profile section
Preconditions	Employee data has been ingested via API
Postconditions	Actor has a holistic view of the employee learning journey

Steps:

18. Actor logs into the system.
19. Actor navigates to the Employee Profiles module.
20. Actor searches for an employee by ID, name, or department.
21. System retrieves and aggregates attendance, assessment, and milestone data.
22. System displays consolidated learning profile with risk and intervention history.
23. Actor can filter and sort by date range, competency area, or status.

UF-02: Risk Rule Configuration

This flow describes how an administrator configures rule-based risk identification.

Field	Details
Flow ID	UF-02
Flow Name	Risk Rule Configuration
Primary Actor	L&D Administrator
Trigger	Need to define or update risk identification rules
Preconditions	Admin has Rule Configuration permissions
Postconditions	New/updated risk rule is active in the system

Steps:

24. L&D Administrator navigates to Risk Rule Configuration.
25. Admin selects Create New Rule or Edit Existing Rule.
26. Admin defines rule name, condition type, thresholds, severity, and status.
27. System validates rule syntax and ranges.
28. Admin saves and activates the rule.
29. System confirms activation for the next evaluation cycle.
30. Admin can review active rules in the rule dashboard.

UF-03: At-Risk Learner Identification & Review

This flow covers automated identification of at-risk learners and administrator review.

Field	Details
Flow ID	UF-03
Flow Name	At-Risk Learner Identification & Review
Primary Actor	System (Automated) + L&D Administrator
Trigger	Scheduled risk evaluation cycle or on-demand trigger
Preconditions	Risk rules are configured and data is current

Postconditions	At-risk learners are identified and ready for intervention
----------------	--

Steps:

31. System executes active risk rules against employee data.
32. System evaluates each employee against applicable rules.
33. Employees meeting risk criteria are flagged with severity.
34. System generates an At-Risk Learner Report.
35. L&D Administrator receives notification.
36. Admin reviews flagged learners.
37. Admin drills into individual profiles for root-cause understanding.
38. Admin initiates intervention assignment.

UF-04: Intervention Assignment & Tracking

This flow describes assigning an intervention and tracking its outcome.

Field	Details
Flow ID	UF-04
Flow Name	Intervention Assignment & Tracking
Primary Actor	L&D Administrator / Trainer
Trigger	At-risk learner identified; intervention needed
Preconditions	Employee is classified as at-risk
Postconditions	Intervention is recorded, tracked, and outcome captured

Steps:

39. Actor selects an at-risk employee.
40. Actor chooses Assign Intervention.
41. Actor selects intervention type: remedial training or coaching/mentoring.
42. For remedial training, actor sets program, schedule, and completion date.
43. For coaching, actor assigns coach/mentor and defines goals/frequency.
44. System records the intervention assignment.
45. Assigned trainer/coach receives notification.
46. Trainer/coach logs session notes, attendance, and progress.
47. Actor records final outcome: successful / partial / unsuccessful.
48. System updates the employee profile with intervention history.

UF-05: Compliance Report Generation

This flow describes generation of compliance-ready reports.

Field	Details
Flow ID	UF-05
Flow Name	Compliance Report Generation
Primary Actor	Compliance Officer / L&D Administrator

Trigger	Regulatory audit, periodic review, or ad-hoc request
Preconditions	Sufficient data is available and user has permissions
Postconditions	Compliance-ready report is generated and exportable

Steps:

49. Actor navigates to Compliance Reports.
50. Actor selects report type.
51. Actor configures parameters such as date range, department, competency area, and compliance standard.
52. System aggregates the required data and generates the report.
53. Actor reviews report on screen.
54. Actor exports report to PDF or Excel.
55. System logs the report generation event.

UF-06: Dashboard & Analytics View

This flow describes how stakeholders consume operational dashboards.

Field	Details
Flow ID	UF-06
Flow Name	Dashboard & Analytics View
Primary Actor	L&D Administrator / HR Manager
Trigger	Actor navigates to dashboard
Preconditions	Data ingestion pipelines are active
Postconditions	Actor has current visibility into learning progress and risk landscape

Steps:

56. Actor logs in and lands on the dashboard.
57. Dashboard displays key metrics such as active learners, risk by severity, attendance rates, average scores, intervention success rates, and competency completion.
58. Actor filters by department, time, or competency area.
59. Actor drills into metrics for details.
60. Dashboard refreshes based on latest available data.

8.3 Sequence Diagrams

The following textual sequence diagrams show the primary interactions between system components.

SD-01: Data Ingestion Flow

Shows how external source systems submit data using APIs and how the platform validates, stores, and audits the events.

Participants: External System, API Gateway, Data Validation, Data Store, Audit Log

```
External System -> API Gateway: POST /api/attendance | /scores | /milestones
API Gateway -> Data Validation: Validate payload schema & business rules
Data Validation -> Data Store: Store validated records
Data Store -> Audit Log: Log ingestion event
API Gateway <- Data Store: 200 OK (with ingestion summary)
```

SD-02: At-Risk Learner Identification

Shows the scheduled rules evaluation process and L&D notification.

Participants: Scheduler, Rule Engine, Data Store, Risk Classification Store, Notification Service

```
Scheduler -> Rule Engine: Trigger risk evaluation cycle
Rule Engine -> Data Store: Fetch active risk rules
Rule Engine -> Data Store: Fetch employee learning data
Rule Engine -> Rule Engine: Evaluate each employee against each rule
Rule Engine -> Risk Classification Store: Store at-risk classifications
Risk Classification Store -> Notification Service: Notify L&D Admin of new at-risk learners
Notification Service -> L&D Admin: Send alert / notification
```

SD-03: Intervention Assignment

Shows creation of interventions and the assignment notification to trainers/coaches.

Participants: L&D Admin, UI/Portal, Intervention Service, Data Store, Notification Service

```
L&D Admin -> UI/Portal: Select at-risk learner and choose intervention type
UI/Portal -> Intervention Service: Create intervention record
Intervention Service -> Data Store: Store intervention details (type, assignee, schedule, goals)
Intervention Service -> Notification Service: Notify assigned trainer/coach
Notification Service -> Trainer/Coach: Send assignment notification
Intervention Service -> UI/Portal: Confirm intervention created
UI/Portal -> L&D Admin: Display confirmation
```

SD-04: Compliance Report Generation

Shows how a report request is processed, formatted, returned, and logged.

Participants: Compliance Officer, UI/Portal, Report Engine, Data Store, Audit Log

Compliance Officer -> UI/Portal: Select report type & parameters

UI/Portal -> Report Engine: Generate report request

Report Engine -> Data Store: Query attendance, scores, milestones, interventions, and risk data

Data Store -> Report Engine: Return aggregated data

Report Engine -> Report Engine: Apply formatting & compliance template

Report Engine -> UI/Portal: Return generated report

UI/Portal -> Compliance Officer: Display report (with export options)

Report Engine -> Audit Log: Log report generation event

8.4 User Restrictions and Rights

Access is governed by role-based access control (RBAC).

Module / Feature	L&D Administrator	Trainer / Coach	Compliance Officer	HR Manager
Employee Profiles	Full Access	Read-Only (Assigned learners only)	Read-Only	Read-Only (Department-scoped)
Risk Rule Configuration	Full Access (Create / Edit / Delete / Activate)	No Access	Read-Only	No Access
At-Risk Learner Dashboard	Full Access	Read-Only (Assigned learners only)	Read-Only	Read-Only (Department-scoped)
Intervention Management	Full Access (Assign / Track / Close)	Create & Edit (assigned interventions)	Read-Only	Read-Only
Compliance Reports	Full Access (Generate / Export)	No Access	Full Access (Generate / Export)	Read-Only (Department-scoped)
System Dashboards	Full Access	Limited (own assignments only)	Read-Only	Read-Only (Department-scoped)
Audit Logs	Read-Only	No Access	Read-Only	No Access
Data Ingestion APIs	Full Access (Configure / Monitor)	No Access	No Access	No Access
User Management	Full Access (Manage roles for Trainers)	No Access	No Access	No Access

Access Level Legend:

- Full Access (Green) — complete read, write, configure, and export permissions.
- Create & Edit / Limited (Blue) — scoped write access only for assigned responsibilities.
- Read-Only (Yellow) — view-only access with no modification privileges.
- No Access (Red) — feature is hidden or inaccessible for that role.

Additional Access Control Rules

- All access is authenticated and session-based.
- Trainer/Coach access is limited to assigned learners and assigned interventions only.
- HR Manager access is scoped to their department or business unit.
- Compliance Officers have broad read access but cannot create, update, or delete records.
- Audit logs are immutable and append-only.
- API access for data ingestion requires API key authentication and IP whitelisting.
- Role assignments are managed by L&D Administrators and are auditable.
- All data export actions are logged for compliance purposes.

9. High Level Architecture

This section describes the recommended architecture for the Corporate Learning Progress, Intervention & Compliance Tracking System, including the proposed technology stack, rationale, alternatives, component orchestration, engineering quality controls, and CI/CD/CT strategy.

9.1 Tech Stack

Layer	Recommended Technology	Purpose
Frontend	React	Build the web-based UI for dashboards, profiles, reports, and administration screens.
Backend / API Layer	Java (Spring Boot)	Implement REST APIs, rule execution, intervention workflows, reporting orchestration, and security.
Database	PostgreSQL	Store operational data, rule definitions, interventions, audit logs, and report metadata.
Caching (Optional)	Redis	Improve performance for frequently accessed dashboard widgets and metadata.
Async / Streaming (Optional)	Kafka or RabbitMQ	Decouple ingestion, evaluation, notification, and report generation at scale.
Object Storage (Optional)	Azure Blob / S3-compatible storage	Persist exported reports, archives, and extract files.
Containerization	Docker	Create environment-consistent application images.
Orchestration	Kubernetes / OpenShift / AKS	Deploy and scale frontend, API, and background jobs.
Observability	Prometheus + Grafana + ELK / OpenSearch	Metrics, logs, and troubleshooting for production operations.
CI/CD Tooling	Azure DevOps or GitHub Actions	Automate build, test, scan, package, deploy, and release governance.

9.2 Why This Tech Stack

Java (Spring Boot)

Well suited for enterprise-grade backend systems due to maturity, stability, strong typing, excellent ecosystem support, transaction management, validation, scheduling, and security capabilities.

React

A strong fit for data-heavy operational UIs with reusable components, efficient rendering, dashboard-centric experiences, and flexible integration patterns.

PostgreSQL

Provides reliable relational modeling, ACID transactions, indexing, JSON support, and strong reporting/query capabilities needed for audit-sensitive operational systems.

Docker + Kubernetes

Enable consistent deployments, autoscaling, rolling releases, and operational resilience as the platform grows.

Optional Async Components

Cache and messaging are not mandatory initially but provide a path to scale without redesigning core flows later.

9.3 Alternatives

Option	Alternative	When to Consider	Trade-Offs
Backend	Node.js / NestJS	When the team is stronger in TypeScript and prefers one language across frontend and backend.	May improve speed for some teams, but Java often remains the stronger enterprise default in tightly governed architectures.
Backend	.NET / ASP.NET Core	When the organization standard is Microsoft-centric.	Excellent enterprise option but diverges from the requested Java-first stack.
Frontend	Angular	When a more opinionated enterprise framework is preferred.	Good structure for large teams, but React is often more flexible and lighter.
Database	MySQL / MariaDB	When operations teams already standardize on it.	Capable, but PostgreSQL typically offers richer SQL/JSON features.
Database	Microsoft SQL Server	When there is an existing SQL Server estate and licensing model.	Strong tooling, but can increase deployment and licensing overhead.
Reporting	Power BI / Superset	When advanced executive analytics are needed beyond built-in operational reporting.	Adds another platform and governance surface.

| Reports / Export Store |
| PDF / Excel / Archive |
+-----+

Supporting capabilities: Security, Audit Logging, Monitoring, Metrics, Traceability

Component Responsibilities

- React Web UI: operational interface for administrators, trainers, HR managers, and compliance officers.
- API Gateway/BFF: ingress for UI and integration clients; can enforce authentication, throttling, and route management.
- Ingestion Service: validates and persists incoming data from attendance, assessment, and milestone sources.
- Profile Service: builds unified employee learning profiles from ingested records.
- Rule Engine: evaluates active rules and classifies at-risk learners on schedule or on demand.
- Intervention Service: manages assignment, lifecycle, and outcomes of remedial training and coaching.
- Reporting Service: generates dashboards, compliance reports, and exportable outputs.
- Notification Service: sends alerts when action is required.
- PostgreSQL: system of record for profiles, rules, risk results, interventions, audit logs, and report metadata.

9.5 How Development, Testing, Deployment, and Code Quality Checks Will Be Ensured

Development Approach

- Use modular service boundaries from the beginning: ingestion, profiles, rules, interventions, reporting, and notifications.
- Adopt API-first design with versioned REST contracts and OpenAPI specifications.
- Maintain separate environments for local development, integration, UAT, and production.
- Use protected branches, peer reviews, and a standard branching strategy (feature branching or trunk-based development with short-lived branches).
- Define coding standards for Java and React, including logging, validation, error handling, and API response conventions.

Testing Strategy

- Unit tests for backend services, utilities, rule evaluators, and frontend components.

- Contract tests for frontend-backend and source-system API compatibility.
- Integration tests for database persistence, ingestion, rules, interventions, and reporting flows.
- End-to-end tests for critical business journeys such as profile view, rule creation, intervention assignment, and report export.
- Performance tests for ingestion throughput, dashboard response, and scheduled evaluation jobs.
- Security testing including SAST, dependency scanning, secret scanning, and DAST on deployed environments.

Deployment Controls

- Use immutable Docker images tagged with commit hash, build number, and version.
- Promote the same artifact across environments instead of rebuilding per environment.
- Manage environment-specific configuration through secure secret stores and config maps.
- Use database migration tooling such as Flyway or Liquibase for consistent, auditable schema changes.
- Enable rolling, blue-green, or canary deployment patterns based on maturity.

Code Quality Checks

- Static analysis for Java using Checkstyle, PMD, SpotBugs, and for React/TypeScript using ESLint and Prettier.
- Code coverage thresholds enforced in CI for critical modules.
- SonarQube quality gates for duplication, maintainability, security hotspots, and code smells.
- Dependency vulnerability scanning using enterprise-approved tools such as Snyk or OWASP Dependency-Check.
- Container image scanning before release.
- Mandatory pull-request review and branch policies before merge.

Recommended Tooling by Stage

Stage	Recommended Tools / Practices
Code	Git, branching strategy, commit standards, PR review
Build	Maven/Gradle for Java, npm/pnpm for React
Unit Test	JUnit, Mockito, AssertJ; Jest/Vitest and React Testing Library
Integration Test	Spring Boot Test, Testcontainers, REST Assured or Postman/Newman
End-to-End Test	Playwright or Cypress

Static Analysis	Checkstyle, PMD, SpotBugs, ESLint, SonarQube
Security	SAST, dependency scanning, secret scanning, container scanning, DAST
Artifact	Docker registry and versioned release artifacts
Deploy	Azure DevOps pipelines / GitHub Actions + Kubernetes manifests / Helm
Observe	Application logs, metrics, traces, dashboards, alerting

9.6 CI / CD / CT Strategy

Use a gated engineering pipeline that combines Continuous Integration (CI), Continuous Delivery/Deployment (CD), and Continuous Testing (CT).

Pipeline Flow

```

Developer Commit / Pull Request
|
v
[CI Stage]  Lint -> Compile -> Unit Tests -> Static Analysis -> Dependency / Secret Scan
|
v
[Package Stage] Build Docker Image -> Tag Artifact -> Push to Registry
|
v
[CT Stage] Integration Tests -> Contract Tests -> E2E Smoke Tests -> Performance Smoke
|
v
[CD to Dev] Deploy to Development Environment
|
v
[CD to QA/UAT] Automated deploy + regression suite + DAST + UAT approval
|
v
[CD to Prod] Manual approval -> Canary / Rolling deployment -> Post-deploy smoke checks
|
v
Monitoring + Alerts + Automated Rollback if health checks / SLOs fail

```

CI Strategy

- Trigger on every pull request and merge to main.
- Run compilation, linting, unit tests, code coverage checks, static analysis, and security scans.
- Fail fast on quality gate violations or vulnerabilities above the agreed severity threshold.
- Publish build artifacts and reports for traceability.

CT (Continuous Testing) Strategy

- Execute integration tests against ephemeral environments or containers using representative test data.
- Use contract tests to validate API compatibility.

- Automate smoke regression for critical business paths: profile view, rule creation, risk evaluation, intervention assignment, and report generation.
- Schedule performance baseline tests for ingestion and reporting workloads.
- Run periodic security tests and compliance validations as release-readiness checks.

CD Strategy

- Use infrastructure-as-code and declarative deployment manifests for repeatability.
- Promote the same tested image across environments to avoid environment drift.
- Require approval gates before production with release notes and change records.
- Perform rolling or canary deployment with health checks and automated rollback criteria.
- Capture post-deployment validation results and production telemetry before completing the release.

Release Governance and Non-Functional Controls

- Define SLOs/SLAs for API latency, rule evaluation duration, dashboard response time, and report generation time.
- Ensure auditability for rule changes, report exports, intervention updates, and privileged access.
- Encrypt data in transit and at rest; manage secrets centrally.
- Use backup/restore and disaster recovery procedures for PostgreSQL and report archives.
- Establish clear ownership for incident response and operational support.

10. Data Model / ER Diagram

This section defines the core entities, relationships, and logical data model required to support learning profile aggregation, risk evaluation, intervention tracking, and compliance reporting.

10.1 Core Design Principles

- Use a normalized relational model for core operational data to preserve integrity and reduce duplication.
- Separate master/reference data (employees, competencies, training programs) from transactional data (attendance, assessments, milestones, interventions).
- Store rule definitions and rule execution results separately so that business logic remains independent from raw source data.
- Retain immutable audit events for ingestion, rule changes, report generation, and intervention updates.

10.2 Core Entities

Entity	Purpose	Key Attributes
Employee	Master record for learner identity	employee_id, employee_code, full_name, email, department_id, manager_id, status
Department	Organizational grouping	department_id, department_name, business_unit
TrainingProgram	Represents a training activity / program	training_id, title, category, mandatory_flag, validity_period
TrainingSession	A scheduled occurrence of a training program	session_id, training_id, trainer_id, start_datetime, end_datetime, mode
AttendanceRecord	Learner attendance for a session	attendance_id, employee_id, session_id, attendance_status, attendance_pct, source_system
Assessment	Assessment metadata	assessment_id, training_id, competency_id, max_score, pass_score
AssessmentResult	Learner score for an assessment	result_id, assessment_id, employee_id, score, percentage, result_status, attempt_no
Competency	Skill / competency tracked by the platform	competency_id, competency_code, competency_name, level_framework
CompetencyMilestone	Milestones within a competency progression	milestone_id, competency_id, milestone_name, milestone_order, due_rule
EmployeeMilestoneStatus	Learner progress against competency milestones	status_id, employee_id, milestone_id, target_date, completion_date, status
RiskRule	Rule definition for at-risk classification	rule_id, rule_name, rule_type, severity, status, effective_from, effective_to

RiskEvaluationResult	Outcome of rule execution per learner	evaluation_id, employee_id, rule_id, triggered_flag, severity, evaluated_at, evidence_json
Intervention	Assigned remedial action for a learner	intervention_id, employee_id, intervention_type, assigned_by, assigned_to, start_date, due_date, status
InterventionLog	Detailed progress / notes for an intervention	log_id, intervention_id, log_date, activity_type, notes, progress_status
ComplianceReportRun	Metadata for generated reports	report_run_id, report_type, generated_by, generated_at, parameter_json, export_path
AuditEvent	Immutable record of sensitive system events	audit_id, event_type, performed_by, performed_at, entity_type, entity_id, details_json

10.3 Relationship Summary

From Entity	Relationship	To Entity
Department	1-to-many	Employee
TrainingProgram	1-to-many	TrainingSession
TrainingSession	1-to-many	AttendanceRecord
Employee	1-to-many	AttendanceRecord
TrainingProgram	1-to-many	Assessment
Assessment	1-to-many	AssessmentResult
Employee	1-to-many	AssessmentResult
Competency	1-to-many	CompetencyMilestone
Employee	1-to-many	EmployeeMilestoneStatus
CompetencyMilestone	1-to-many	EmployeeMilestoneStatus
RiskRule	1-to-many	RiskEvaluationResult
Employee	1-to-many	RiskEvaluationResult
Employee	1-to-many	Intervention
Intervention	1-to-many	InterventionLog
ComplianceReportRun	1-to-many	AuditEvent (optional link)

10.4 Textual ER Diagram

```

Department (1) ----- (M) Employee
Employee (1) ----- (M) AttendanceRecord --- (M:1) TrainingSession ---
(M:1) TrainingProgram
Employee (1) ----- (M) AssessmentResult --- (M:1) Assessment --- (M:1)
TrainingProgram
Competency (1) ----- (M) CompetencyMilestone
Employee (1) ----- (M) EmployeeMilestoneStatus --- (M:1)
CompetencyMilestone
RiskRule (1) ----- (M) RiskEvaluationResult --- (M:1) Employee
Employee (1) ----- (M) Intervention --- (1:M) InterventionLog
AuditEvent captures system changes across ingestion, rule updates, interventions, and
reports

```

10.5 Recommended Indexing and Data Governance

- Index employee_id, department_id, training_id, competency_id, evaluated_at, and generated_at for common query patterns.

- Use soft-delete or status-based lifecycle controls rather than hard delete for audit-sensitive business entities.
- Partition large transactional tables (e.g., AuditEvent, RiskEvaluationResult, AttendanceRecord) by date or reporting period if volumes grow significantly.
- Maintain source-system identifiers to support traceability and idempotent ingestion.

11. Rule Definition Format

This section defines a practical, implementation-friendly format for representing rule-based risk identification logic. The goal is to keep rule configuration business-friendly while making evaluation deterministic and auditable.

11.1 Rule Design Principles

- Rules must be data-driven and configurable without code changes.
- Each rule must clearly identify its target population, triggering conditions, severity, and effective period.
- Rules must support simple threshold conditions as well as composite conditions.
- Every rule evaluation must leave behind explainable evidence to support trainer action and compliance audits.

11.2 Recommended Rule Structure

Field	Description
rule_id	Unique system-generated identifier for the rule
rule_name	Business-friendly name of the rule
description	Explanation of the risk logic and purpose
target_scope	Population to which the rule applies (all learners, department, training program, competency, region, etc.)
rule_type	Attendance, Assessment, Milestone, Composite, or Custom
severity	Low, Medium, High, Critical
effective_from / effective_to	Validity window for the rule
status	Draft, Active, Inactive, Retired
conditions	One or more conditions with operators, thresholds, and evaluation window
threshold_window	Time window to evaluate over (e.g., last 30 days, current quarter)
action_guidance	Recommended action when the rule triggers
created_by / approved_by	Governance metadata for changes and approvals

11.3 Example JSON Format

```
{
  "rule_name": "Low Attendance for Mandatory Training",
  "description": "Flag learners whose attendance is below threshold for mandatory sessions.",
  "target_scope": {
    "applies_to": "training_program",
    "training_category": "mandatory"
  },
  "rule_type": "Attendance",
  "severity": "High",
  "threshold_window": "last_30_days",
  "status": "Active",
```

```

"conditions": [
  {
    "metric": "attendance_percentage",
    "operator": "<",
    "value": 75
  }
],
"action_guidance": "Assign remedial training and notify trainer"
}

```

11.4 Supported Rule Types

Rule Type	Description	Example
Attendance	Evaluates attendance participation	Attendance < 75% in mandatory sessions
Assessment	Evaluates score or pass/fail performance	Latest assessment score < 60%
Milestone	Evaluates missed or delayed progression	Milestone not completed within 10 days of target date
Composite	Combines multiple conditions	Attendance < 80% AND assessment < 65%
Custom	Reserved for future advanced or expression-based evaluation	Department-specific compliance logic

11.5 Sample Rules

Sample Rule 1: Low Attendance Risk

- Condition: attendance_percentage < 75
- Window: current_month
- Severity: High
- Suggested Action: assign remedial training and notify L&D administrator

Sample Rule 2: Poor Assessment Performance

- Condition: latest_assessment_percentage < 60
- Window: current_cycle
- Severity: High
- Suggested Action: assign coach and retest plan

Sample Rule 3: Missed Milestone Deadline

- Condition: completion_date is null AND current_date > target_date + 10 days
- Window: rolling
- Severity: Medium
- Suggested Action: escalate to trainer / manager

Sample Rule 4: Composite Risk

- Condition: attendance_percentage < 80 AND latest_assessment_percentage < 65
- Window: current_quarter
- Severity: Critical
- Suggested Action: remedial training + coaching + manager notification

11.6 Rule Evaluation Output

- employee_id and employee context
- rule_id and rule_name
- triggered_flag (true/false)
- severity
- evaluation_timestamp
- evidence payload showing metrics, threshold, actual values, and relevant period
- recommended action / intervention suggestion

12. API Specifications

This section defines the primary REST API surfaces needed to ingest data, retrieve learner profiles, manage interventions, and support reporting and administration.

12.1 API Design Principles

- RESTful, versioned APIs (e.g., /api/v1/...)
- JSON request / response format
- Idempotency support for ingestion endpoints where duplicate submissions are possible
- Standard error model with validation details and trace identifiers
- Authentication and authorization enforced consistently across endpoints

12.2 Core Endpoint Summary

Category	Method	Endpoint	Purpose
Ingestion	POST	/api/v1/attendance	Ingest attendance records
Ingestion	POST	/api/v1/assessment-results	Ingest assessment scores/results
Ingestion	POST	/api/v1/milestones	Ingest competency milestone progress
Profiles	GET	/api/v1/employees/{employeeId}/profile	Retrieve consolidated employee learning profile
Profiles	GET	/api/v1/employees	Search / filter employees
Risk Rules	GET	/api/v1/risk-rules	List rules
Risk Rules	POST	/api/v1/risk-rules	Create a new risk rule
Risk Rules	PUT	/api/v1/risk-rules/{ruleId}	Update an existing rule
Risk Rules	POST	/api/v1/risk-rules/{ruleId}/activate	Activate a rule
Risk Evaluation	POST	/api/v1/risk-evaluations/run	Trigger on-demand rule evaluation
Risk Dashboard	GET	/api/v1/at-risk-learners	Fetch at-risk learner list
Interventions	POST	/api/v1/interventions	Assign a new intervention
Interventions	GET	/api/v1/interventions/{interventionId}	Get intervention details
Interventions	POST	/api/v1/interventions/{interventionId}/logs	Add intervention log entry
Reports	POST	/api/v1/reports/compliance	Generate compliance report
Reports	GET	/api/v1/reports/{reportRunId}	Fetch generated report metadata/status

12.3 Sample Request / Response Shapes

Attendance Ingestion Example

```
POST /api/v1/attendance
{
```

```

"source_system": "LMS-A",
"records": [
  {
    "employee_code": "E1001",
    "session_id": "S2001",
    "attendance_status": "Present",
    "attendance_percentage": 100,
    "attended_at": "2026-05-20T10:00:00Z"
  }
]
}
{
  "status": "SUCCESS",
  "accepted_records": 1,
  "rejected_records": 0,
  "trace_id": "abc-123"
}

```

Employee Profile Response Example

```

GET /api/v1/employees/{employeeId}/profile
{
  "employee_id": 101,
  "employee_name": "John Doe",
  "department": "Operations",
  "attendance_summary": { "mandatory_training_pct": 82 },
  "assessment_summary": { "latest_average_pct": 68 },
  "milestone_summary": { "completed": 4, "pending": 2 },
  "current_risk_status": "High",
  "active_interventions": 1
}

```

Intervention Assignment Example

```

POST /api/v1/interventions
{
  "employee_id": 101,
  "intervention_type": "COACHING",
  "assigned_to": 901,
  "goals": ["Improve score to 75%+", "Complete missed milestone"],
  "due_date": "2026-06-15"
}

```

12.4 Standard Error Model

```

{
  "status": "ERROR",
  "error_code": "VALIDATION_FAILED",
  "message": "One or more request fields are invalid.",
  "trace_id": "abc-123",
  "details": [
    { "field": "attendance_percentage", "issue": "must be between 0 and 100" }
  ]
}

```

12.5 Security and API Governance

- Use OAuth2 / JWT or enterprise-approved service authentication for user-driven APIs.

- Use API keys plus IP whitelisting and idempotency headers for source-system ingestion APIs.
- Apply rate limiting, request validation, and trace IDs across all endpoints.
- Version APIs and deprecate changes with explicit backward compatibility planning.

13. Non-Functional Requirements (NFRs)

This section captures the non-functional expectations required to operate the platform reliably, securely, and in a compliance-sensitive enterprise environment.

13.1 Security

- Role-based access control must be enforced for all UI features and APIs.
- All data must be encrypted in transit using TLS and encrypted at rest using platform-approved mechanisms.
- Sensitive actions such as rule changes, exports, intervention edits, and access-right changes must be auditable.
- Secrets must be stored in a secure secrets manager and never hard-coded in source repositories or pipelines.

13.2 Performance

- Profile retrieval APIs should respond within acceptable operational SLAs for normal usage patterns.
- Dashboard widgets should load quickly and support caching where appropriate.
- Rule-evaluation jobs should complete within an agreed batch window for the expected employee population.
- Report generation should provide progress status and support asynchronous processing for large extracts.

13.3 Scalability

- The architecture should support horizontal scaling of stateless services such as API, reporting, and rule evaluation workers.
- Data model and indexing strategy should support increasing employee volumes, more source systems, and longer audit retention periods.
- Optional queue / cache components should be introducible without changing business interfaces.

13.4 Availability and Reliability

- Production deployments should support rolling or canary release patterns to reduce downtime.
- Database backup, restore, and disaster recovery procedures must be defined and periodically validated.

- Critical scheduled jobs and integrations should have retry, alerting, and failure visibility mechanisms.

13.5 Auditability and Compliance

- Every risk-rule change must be traceable to the user, time, and before/after values.
- Every report export and intervention update must be recorded in immutable audit logs.
- Source-system ingestion must preserve lineage to the original record or source event.

13.6 Observability and Supportability

- Application logs, metrics, and traces must be centralized and searchable.
- Alerts should be configured for failed ingestion, failed rule jobs, report generation failures, and service health degradation.
- Operational dashboards should expose high-level health indicators for APIs, jobs, and integrations.

13.7 Maintainability

- Code should be modular, testable, and aligned with agreed coding standards.
- Database schema changes should be migration-driven and version controlled.
- APIs should be documented using OpenAPI and changes governed via version control and release notes.